

AUTONOMOUS SOFTWARE EXPERIMENTS ON HERA: ANNEX C - HERA CLIENT STUB, USER & INTEGRATION GUIDE

Reference	ESA-HERA-TECS-MAN-2026-002051
Issue/Revision	1.0
Date of Issue	19/06/2026

Table of Contents

1. Overview	3
2. File Structure	3
3. Key Functionalities	3
3.1. Simulated Delays (Blocking Behaviour)	3
3.2. Image Sequence Simulation (AFC).....	3
3.2.1. How it works.....	4
3.2.2. Use Case	4
3.3. UART Console Output.....	4
4. Customizing Test Images	5
5. Compilation	5
6. Running with QEMU	5
6.1. Flags explanation	5
6.2. To exit QEMU	5

1. OVERVIEW

The Hera Client Stub (`hera_client_stub.c`) is a software simulation layer designed to mimic the behaviour of the real Hera Payload Interface running on Core 1 of the LEON3 processor.

It allows developers to:

- Develop and test experiment software (AFC, HyperScout, TIRI, Reporting) on a local PC using QEMU.
- Simulate hardware delays (e.g., acquisition times) without real hardware.
- Feed test data (sequence of images) to image processing algorithms to verify logic.
- Debug via UART using a lightweight, bare-metal print implementation.

2. FILE STRUCTURE

To use the stub, ensure the following files are present in the source directory:

- `hera_interface.h`: The public API definition (identical to the flight version).
- `hera_client_stub.c`: The simulation implementation.
- `stub_utils.h`: Internal helpers for the stub (UART, formatting).
- `images_data.h`: Database of test images.
- `your_main.c`: Your application code (the “Experiment Software”).

3. KEY FUNCTIONALITIES

3.1. Simulated Delays (Blocking Behaviour)

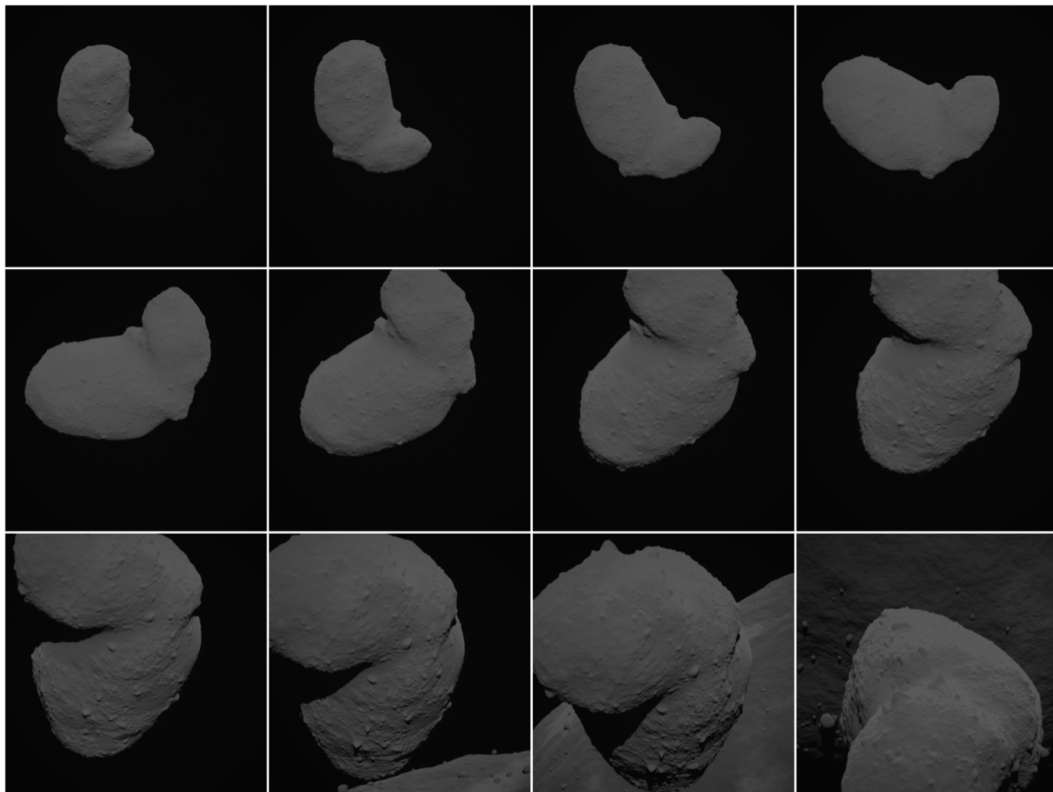
On the real spacecraft, functions like `Hera_AFC_AcquireSingleImage` block the CPU while Core 0 communicates with hardware.

The stub simulates this by entering a “busy wait” loop.

- AFC Acquisition: Sleeps for 7 seconds (simulated).
- HS Acquisition: Sleeps for 15 minutes (simulated).
- Store operations: Sleeps for 10 seconds (simulated).

3.2. Image Sequence Simulation (AFC)

This feature provides a robust way to test image processing algorithms. Instead of returning static noise or a single image, the stub implements a circular image buffer.



3.2.1. How it works

1. Multiple images are defined in `images_data.h` (e.g., Image 1, Image 2, Image 3).
2. First call to `Hera_AFC_AcquireSingleImage()` loads Image 1.
3. Second call loads Image 2.
4. Third call loads Image 3.
5. Fourth call loops back and loads Image 1.

3.2.2. Use Case

You can write a loop in your `main.c` that acquires an image, processes it, and repeats. The stub will automatically feed your algorithm different test scenarios sequentially.

3.3. UART Console Output

UART support is provided only in the stub for debugging. The real HERA interface does not allow any communication over UART.

- Output: All stub debug messages (e.g., `[STUB] Sleeping...`) are sent via UART and can be shown on STDOUT.

- Input: Your code should use `print_str()`, `print_dec()`, and `print_hex()` (exposed via `stub_utils.h` if needed for debugging).

4. CUSTOMIZING TEST IMAGES

To test your specific algorithms, replace the dummy data in `images_data.h` with your own raw image bytes:

1. Open `images_data.h`.
2. Update the `image1`, `image2`, etc. arrays with your raw byte data (hex dump). See Annex D for details on a Python tool to generate `images_data.h`.
3. If adding more images, update `NUM_TEST_IMAGES` and add the new array to the `test_images_bank` array.

5. COMPILATION

You must compile your main application linked together with the stub implementation.

```
sparc-gaisler-elf-gcc -O2 -g your_main.c hera_client_stub.c -o experiment_test.elf
```

- `-O2`: Optimisation level 2.
- `-g`: Include debug symbols (useful if using `gdb`).
- `-o`: Output filename.

6. RUNNING WITH QEMU

To execute the compiled binary, use the QEMU emulator configured for the LEON3 architecture:

```
qemu-system-sparc -M leon3_generic -kernel experiment_test.elf -nographic
```

6.1. Flags explanation

- `-M leon3_generic`: Emulates a generic LEON3 board.
- `-kernel experiment_test.elf`: Loads your compiled program as the kernel.
- `-nographic`: Disables the QEMU video window and redirects the emulated UART (serial port) to your terminal. This allows you to see the `print_str` output.

6.2. To exit QEMU

- Press `Ctrl+A`, release, then press `x`.